Activity – 3 :

Simple Agent Function for the Vacuum-Cleaner World

Submitted by: D. Jeshurun Silas
Registration Number: 192525324
Type: Activity Report
Subject Area: Artificial Intelligence – Intelligent Agents

1. Abstract

This report presents the design of a simple agent function for the classic two-location vacuum-cleaner world. The agent perceives its current location and whether that location is dirty, and chooses between cleaning, moving left, or moving right. The report describes the task environment using the PEAS framework, defines the agent function as a lookup table, converts it into a simple reflex agent program, provides a Python implementation with a sample simulation, and discusses performance and limitations.

2. Introduction

An **agent** is anything that perceives its environment through sensors and acts upon it through actuators. The **agent function** is the abstract mapping from the percept sequence (everything the agent has perceived so far) to an action. The **agent program** is the concrete implementation of that function running on a physical architecture.

The vacuum-cleaner world, popularised by Russell and Norvig, is a deliberately small environment used to explain these ideas. It has just two squares, **A** and **B**, each of which may be clean or dirty. Despite its simplicity, it demonstrates percepts, actions, rationality, performance measures and agent types clearly.

3. Objectives

- To define the vacuum-cleaner world and its task environment.
- To specify a simple agent function for this world.
- To implement the function as a simple reflex agent program.
- To evaluate the rationality and limitations of the design.

4. Task Environment (PEAS Description)

Component	Description
Performance measure	One point for each clean square at each time step (optionally with a penalty for each move or energy used)
Environment	Two squares, A and B, each either Clean or Dirty; the agent starts in a randomly chosen square
Actuators	Move Left, Move Right, Suck (clean the current square)
Sensors	Location sensor (A or B) and

Component	Description
	dirt sensor (Clean or Dirty for the current square only)

4.1 Environment Properties

Property	Classification	Reason
Observability	Partially observable	The agent senses only its current square, not the other one
Agents	Single-agent	Only the vacuum cleaner acts
Determinism	Deterministic	Actions have predictable outcomes
Episodic vs. sequential	Sequential	Current actions affect future states
Static vs. dynamic	Static	Dirt does not appear on its own while the agent decides
Discrete vs. continuous	Discrete	Finite locations, percepts and actions
Known vs.	Known	The rules and

Property	Classification	Reason
unknown		outcomes of actions are known

5. Percepts, Actions and State Space

Percepts: [Location, Status] where Location ∈ {A, B} and Status ∈ {Clean, Dirty}.

Actions: {Left, Right, Suck, (optionally NoOp)}.

The world has 2 × 2 × 2 = **8 possible states** (agent location × status of A × status of B).

6. The Agent Function

The agent function maps a percept to an action. A partial tabulation is shown below.

Percept sequence	Action
[A, Clean]	Right
[A, Dirty]	Suck
[B, Clean]	Left
[B, Dirty]	Suck
[A, Clean], [A, Clean]	Right
[A, Clean], [B, Dirty]	Suck
…	…

Because the table over full percept sequences grows without bound, it is impractical. However, the correct action depends

only on the **current percept**, so the whole table collapses into four simple rules.

6.1 Condition–Action Rules

1. If the current square is **Dirty**, then **Suck**.
2. Else if the location is **A**, then move **Right**.
3. Else if the location is **B**, then move **Left**.

7. Agent Program (Simple Reflex Agent)

A simple reflex agent selects actions based only on the current percept, ignoring history.

7.1 Pseudocode

```
function REFLEX-VACUUM-AGENT([location, status]) returns an action
    if status = Dirty then return Suck
    else if location = A then return Right
    else if location = B then return Left
```

7.2 Python Implementation

```
def reflex_vacuum_agent(percept):
    location, status = percept
    if status == "Dirty":
        return "Suck"
    elif location == "A":
        return "Right"
    else:
        return "Left"
```

```
def simulate(location, state, steps=6):
    # state example: {"A": "Dirty", "B": "Dirty"}
    score = 0
    for t in range(1, steps + 1):
        percept = (location, state[location])
        action = reflex_vacuum_agent(percept)
        print(f"t={t}: at {location}, sees {state[location]:5} ->
{action}")
        if action == "Suck":
            state[location] = "Clean"
        elif action == "Right":
            location = "B"
        elif action == "Left":
            location = "A"
        score += sum(1 for s in state.values() if s == "Clean")
    print("Final state:", state, "| Score:", score)


simulate("A", {"A": "Dirty", "B": "Dirty"})
```

8. Sample Execution Trace

Starting at A with both squares dirty:

Time step	Location	Percept	Action	Resulting state	Cumulative score
1	A	[A, Dirty]	Suck	A clean, B dirty	1
2	A	[A, Clean]	Right	Agent moves to	2

Time step	Location	Percept	Action	Resulting state	Cumulative score
				B	
3	B	[B, Dirty]	Suck	A clean, B clean	4
4	B	[B, Clean]	Left	Agent moves to A	6
5	A	[A, Clean]	Right	Agent moves to B	8
6	B	[B, Clean]	Left	Agent moves to A	10

Both squares are cleaned by step 3, after which the agent keeps oscillating between A and B.

9. Rationality Analysis

Rationality depends on four things: the performance measure, the agent's prior knowledge of the environment, the available actions, and the percept sequence to date.

- **If the performance measure only rewards clean squares per time step** and dirt never reappears, the agent is rational: it cleans every dirty square it finds and never makes a harmful move.

- **If each movement costs a penalty**, the agent becomes **irrational** after both squares are clean, because it keeps moving needlessly. A better agent would choose **NoOp** once it knows everything is clean.
- **If dirt can reappear**, continued patrolling is reasonable, and the oscillating behaviour is actually useful.

10. Limitations

- **No memory:** the agent cannot remember that the other square is already clean, so it cannot decide to stop.
- **Partial observability:** it only senses the current square.
- **Wasted effort:** it may loop indefinitely with no benefit in a static world with a movement penalty.
- **Poor scalability:** in larger grids, fixed condition–action rules are insufficient and it can get stuck in loops.

11. Possible Improvements

1. **Model-based reflex agent:** keep an internal state recording the last known status of each square, and issue NoOp when both are believed clean.
2. **Goal-based agent:** pursue the explicit goal "all squares clean".
3. **Utility-based agent:** weigh cleanliness against energy and time costs.
4. **Randomised or learning agents:** use randomisation or reinforcement learning for larger or unknown environments.

12. Conclusion

The vacuum-cleaner world shows how an agent function can be defined precisely and then implemented compactly as a simple reflex agent. The four-rule program is rational under a basic performance measure and fully captures the percept-to-action mapping for this small world. Its limitations, notably the lack of memory and its unnecessary movement, motivate richer agent designs such as model-based, goal-based and utility-based agents.

13. References

1. Russell, S. and Norvig, P. *Artificial Intelligence: A Modern Approach*, 4th Edition, Pearson, 2021.
2. Poole, D. and Mackworth, A. *Artificial Intelligence: Foundations of Computational Agents*, 3rd Edition, Cambridge University Press, 2023.